IncidentPilot

An Agentic, Evidence-Grounded Incident Investigation System

Gracy Patel | MS Computer Science (AI/ML), University of Southern California | **Status:** Planned

OVERVIEW

IncidentPilot is an AI agent that investigates production incidents the way an on-call engineer would. Given an alert (e.g., *"checkout latency spiked"*), it plans an investigation, gathers evidence from logs, metrics, deploy history, and runbooks, and produces a root-cause report in which **every claim cites verifiable evidence**. A human approves any recommended fix before the report is finalized.

PROBLEM AND USERS

- **Problem:** Incident evidence is scattered across tools, investigations are slow under pressure, and LLM answers without evidence cannot be trusted in production.
- **Users:** On-call engineers, SREs, DevOps and backend teams.
- **Goal:** An agent that is both *useful* (finds the root cause) and *trustworthy* (no citation, no claim; humans approve actions).

SYSTEM DESIGN

- **Planner agent:** Decomposes the alert into ordered investigation steps.
- **Tools:** Metrics anomaly detection (rolling z-score), time-windowed log search, deploy-history lookup, and runbook retrieval (RAG over ChromaDB with local embeddings).
- **Analyst agent:** Forms a hypothesis; outputs a Pydantic-validated report where each claim lists evidence IDs.
- **Validator:** Rejects claims with missing or non-existent citations and checks timeline consistency (cause must precede symptom).
- **Human-in-the-loop:** LangGraph `interrupt` pauses execution for approval before a fix is recommended.
- **Observability & routing:** Every step (tool, input, output, latency, tokens, cost) is traced to SQLite; lighter models handle simple steps and a stronger model handles final reasoning.

DATA: FAULT-INJECTION SIMULATOR

Real production logs are private, so a custom simulator generates synthetic logs, metrics, and deploy records for a small multi-service system (web, cart, checkout, database, cache). Each incident has one **injected fault** (e.g., bad deploy, memory leak, dependency outage, disk full, cache failure), optional **distractors** (unrelated noise), and a hidden **answer key** with the true root cause and key evidence.

EVALUATION PLAN

- **Metrics:** Root-cause accuracy, citation precision, evidence recall, robustness to distractors, tool calls per investigation, latency, and cost (with vs. without model routing).
- **Baselines:** (1) single LLM call with all evidence in one prompt, (2) agent without the validator, (3) full IncidentPilot.
- **Rule:** Only measured results are reported; no estimated or invented metrics.

TECH STACK

Core:	Python 3.11+, LangGraph, Groq (LLM inference), Pydantic
Retrieval:	Sentence Transformers (local embeddings), ChromaDB
Ops & UI:	SQLite (execution traces), Streamlit (dashboard), Pytest (testing), Git/GitHub
From scratch:	Fault simulator, z-score anomaly detector, log search, citation validator, tracer, and evaluation harness (*fundamentals over frameworks*)

MILESTONES

(1) **MVP:** one fault type investigated end to end with cited evidence; (2) **Simulator:** five fault types with distractors and answer keys; (3) **Quality:** validator, human approval, tracing, and routing; (4) **Evaluation & demo:** benchmark vs. baselines, Streamlit dashboard, README, and demo video.

LIMITATIONS AND FUTURE WORK

Synthetic incidents are simpler than real ones (mitigated with distractors); each has a single root cause; the validator checks that citations exist, not that reasoning is flawless. Future: multi-cause incidents, live log/metric connectors.